

Chapter 3 - Functions

With functions, we can store pieces of code. This allows us to execute this code whenever we want. Functions are also known as methods.

There are 2 ways to create a function:

```
example = function ()  
    print("Hello World!")  
end
```

and the more common way:

```
function example()  
    print("Hello World!")  
end
```

You start by writing the keyword function, followed by the name of the function. A function is a type of variable, so the same rules apply as when you name a variable. This function's name is example. After the name we put parentheses (). Now we can start typing the code we want to put inside our function. In this case, we put in print("Hello World!") When you're done you close the function with an end.

Note that when you run the code, you'll see no "Hello World!" in your console, this is because we still have to execute the function. We execute a function like this:

```
example()
```

--Output: "Hello World!"

You simply type the function's name, followed by parentheses. This is what we call a *function-call*.

Parameters

Take a look at this code:

```
function sayNumber(num)  
    print("I like the number " .. num)  
end
```

```
sayNumber(15)
```

```
sayNumber(2)
```

```
sayNumber(44)
```

```
sayNumber(100)
```

```
print(num)
```

```
--Output:
```

```
--"I like the number 15"
```

```
--"I like the number 2"
```

```
--"I like the number 44"
```

```
--"I like the number 100"
```

```
--nil
```

Inside the parentheses of the function we can put what we call *parameters*. Parameters are temporary variables that only exist inside the function. In this case we place the parameter `num`. And now we can use `num` like any other variable.

We execute our function multiple times, each time with a different number. And thus each time we print the same sentence, but with a different number. The number we put inside the parentheses is what we call an argument. So in the first function-call, we *pass* the *argument* 15 to the *parameter* `num`.

At the end of our code we print `num`, outside of our function. This gives us `nil`. This means that `num` has no value. It's not a number, or string, or function. It's nothing. Because like I said before, parameters are variables that are only available inside the function.

Return

Functions can return a value, which we can store inside a variable, for example. You can return a value by using the `return` keyword.

```
function giveMeFive()
```

```
  return 5
```

end

```
a = giveMeFive()
```

```
print(a)
```

--Output: 5

a becomes the value that *giveMeFive* *returns*.

Another example:

-- Multiple parameters and arguments are separated by a comma

```
function sum(a, b)
```

```
    return a + b
```

end

```
print(sum(200, 95))
```

--Output:

--295

Our function *sum* *returns* the sum of a and b. We don't necessarily need to put the value our function returns in a variable first. We can directly print the value.

Usage

Often you want to execute certain code in multiple locations. Instead of copying that code each time you want to use it, we can simply add a function call. And if we want to change the behaviour of this function, we only need to change it in one location, which is the function itself. This way we avoid repeating code. [Don't repeat yourself](#), it's one of the most important programming principles.

Summary

Functions can store code that we can execute at any time. We call a function by writing its name followed by parentheses. We can put values inside these parentheses. These values

are passed to the function's parameters, which are temporary variables that only exist within the function. Functions can also return a value. Functions remove repetition and that is a good thing.